

Supervised Classification: The Prediction Task

Maxime Boucher¹ – Yannick Voglaire¹

¹ICHEC - Brussels Management School,
Laboratoire QUARESMI
Site d'Anjou, Rue au Bois, 365a
QUARESMI : bureau B307
1150 Bruxelles

Classification



- 1 Introduction
- 2 Main Algorithms
- 3 Validation



- 1 Introduction
- 2 Main Algorithms
- 3 Validation



1 Introduction

- Supervised Classification is Prediction
- A First Algorithm: k-Nearest Neighbours
- A First Evaluation Procedure



Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
4.8	3.1	1.6	0.2	setosa
5.7	3.8	1.7	0.3	setosa
4.6	3.1	1.5	0.2	setosa
5.5	2.4	3.8	1.1	versicolor
5.7	2.8	4.5	1.3	versicolor
5.1	2.5	3.0	1.1	versicolor
7.9	3.8	6.4	2.0	virginica
6.7	2.5	5.8	1.8	virginica
6.2	2.8	4.8	1.8	virginica
7.1	3.0	5.9	2.1	virginica



Principles:

- Given two sets of variable $\mathcal{X}$ and Y , how to predict the value $y \in Y$ for a given $x \in \mathcal{X}$?
- Building a prediction function f and estimate y using f : $\hat{y} = f(x)$.
- Using sample $u_1, \dots, u_n$, called the training set, for which the couples (x_i, y_i) are both known.
- x are called explicative variables.
- y are called target variables.

Applications:

- Regressions: when y is a value in R^d ;
- **Discrimination/Classification: when y is the belonging to a group.**

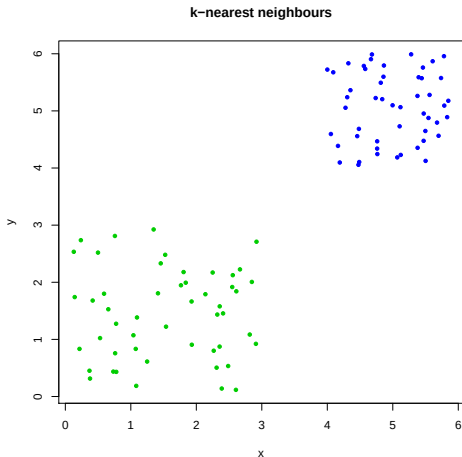


1 Introduction

- Supervised Classification is Prediction
- A First Algorithm: k-Nearest Neighbours
- A First Evaluation Procedure



The most Simple: k-nearest neighbours



For

- a chosen distance measure d ,
- a chosen k parameter,
- a given individual u to classify.

The algorithm is the following:

- compute the distance between u and all data in the training dataset,
- select the k nearest neighbours of u in the training dataset,
- the class of the u is the majority class these k nearest neighbours.

Drawback: choice of d and choice of k , computing all the distances for each new individual to class, normalization of variables.



1 Introduction

- Supervised Classification is Prediction
- A First Algorithm: k-Nearest Neighbours
- A First Evaluation Procedure



Training Set and Test Set



Training Set and Test Set

- Divide the set of known groups in two sets:



Training Set and Test Set

- Divide the set of known groups in two sets:
 - ① the Training set,



Training Set and Test Set

- Divide the set of known groups in two sets:
 - 1 the Training set,
 - 2 the Test set.



Training Set and Test Set

- Divide the set of known groups in two sets:
 - 1 the Training set,
 - 2 the Test set.
- Use the Training set to “tune the model”.



Training Set and Test Set

- Divide the set of known groups in two sets:
 - 1 the Training set,
 - 2 the Test set.
- Use the Training set to “tune the model”.
- Predict the groups for the Test set and compare with the true values.



Question : How to measure the quality of a predictive model?



Question : How to measure the quality of a predictive model?

The starting point: the confusion matrix

	Predicted Positive	Predicted Negative
Really Positive	TP	FN
Really Negative	FP	TN

Remark:

The confusion matrix is defined for a two classes problem but can be extended for more classes.



	Predicted Positive	Predicted Negative
Really Positive	TP	FN
Really Negative	FP	TN

Measures:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad \% \text{ of total good predictions}$$

$$\text{Error Rate} = \frac{FP + FN}{TP + TN + FP + FN} \quad \% \text{ of bad predictions}$$



- 1 Introduction
- 2 Main Algorithms
- 3 Validation

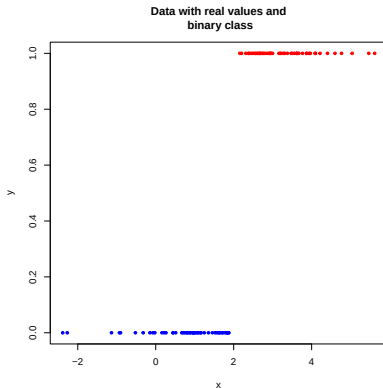


2 Main Algorithms

- Logistic Regression
- Bayesian Supervised Classification
- Support Vector Machine - SVM
- Decision Trees

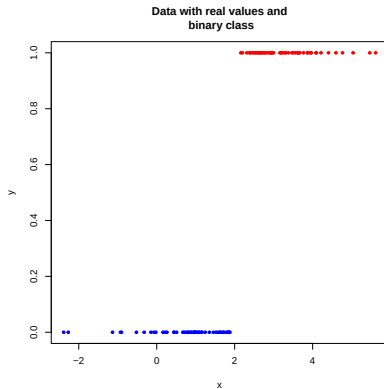


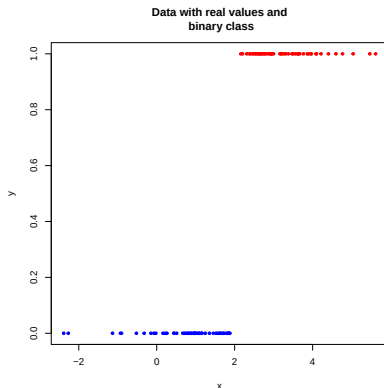
Let a dataset with two variables y and x where $x \in \mathbb{R}$ and $y \in \{0, 1\}$.



And suppose that we wish to explain y from $x = y = f(x)$.







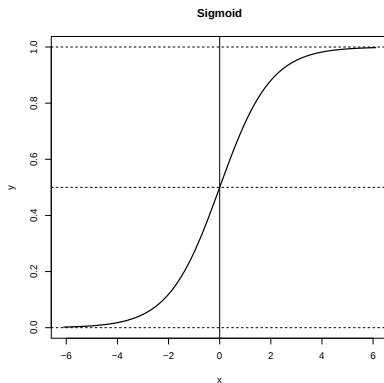
A probability function such $p(x_i) = P(y = 1|x_i)$ should be great!
And of course $1 - p(x_i) = P(y = 0|x_i)$.

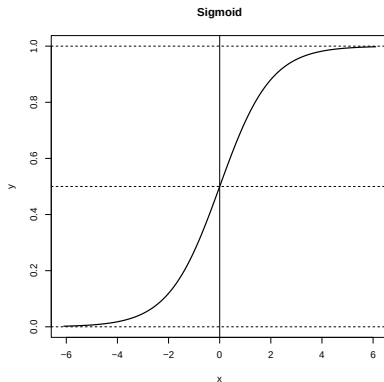


$$p(x) = f_s(x) = \frac{1}{1 + e^{-x}}$$



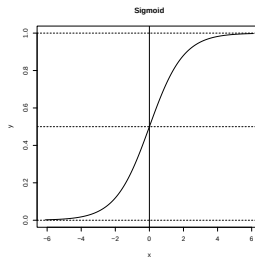
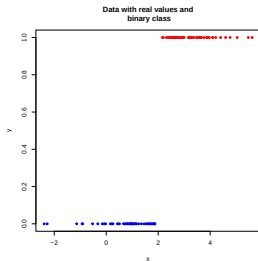
$$p(x) = f_s(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{1 + e^x}$$

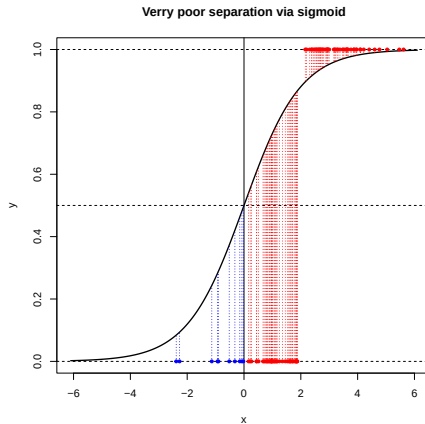


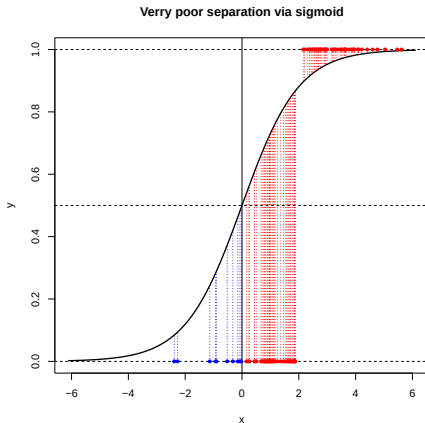


$$y_i = \begin{cases} 1 & \text{if } p(x_i) > 0.5 \\ 0 & \text{if } p(x_i) \leq 0.5 \end{cases}$$



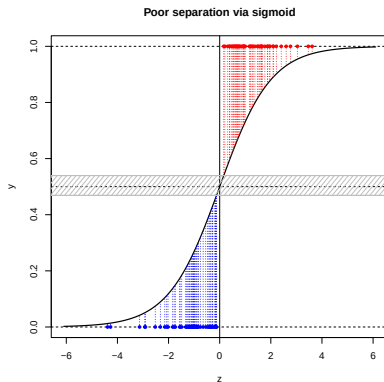


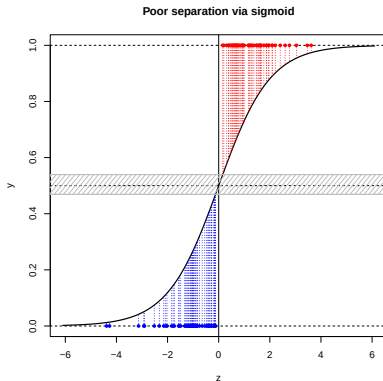




It will be okay if, instead of $f_s(x)$ we compute $f_s(z)$
with $z = \beta_0 + x$ (here $\beta_0 \approx -0.4$).







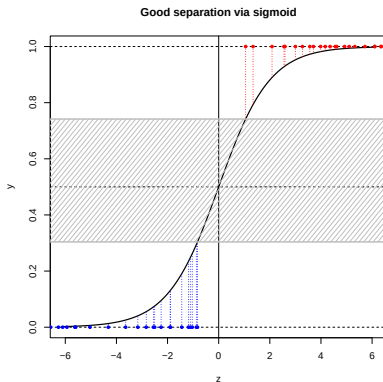
...the safety margin around 0.5 is too small.



It will be okay if we "rescale" x before adding β_0 , let's compute :

$$f_s(z) \text{ with } z = \beta_0 + \beta_1 \cdot x$$

here $\beta_1 \approx 0.15$.



It's better !

So

$$\begin{aligned} P(y = 1|x_i) = p(x_i) = f_s(z_i) &= \frac{e^{z_i}}{1 + e^{z_i}} \\ &= \frac{e^{\beta_0 + \beta_1 \cdot x_i}}{1 + e^{\beta_0 + \beta_1 \cdot x_i}} \end{aligned}$$

And of course

$$\begin{aligned} P(y = 0|x_i) = 1 - p(x_i) &= 1 - \frac{e^{z_i}}{1 + e^{z_i}} \\ &= \frac{1}{1 + e^{z_i}} \\ &= \frac{1}{1 + e^{\beta_0 + \beta_1 \cdot x_i}} \end{aligned}$$



If

$$p(x_i) = \frac{e^{\beta_0 + \beta_1 \cdot x_i}}{1 + e^{\beta_0 + \beta_1 \cdot x_i}} \quad \text{and} \quad 1 - p(x_i) = \frac{1}{1 + e^{\beta_0 + \beta_1 \cdot x_i}}$$

then

$$\frac{p(x_i)}{1 - p(x_i)} = e^{\beta_0 + \beta_1 \cdot x_i}$$

and thus

$$\log \left(\frac{p(x_i)}{1 - p(x_i)} \right) = \underbrace{\beta_0 + \beta_1 \cdot x_i}_{\text{linear part}}$$



If

$$p(x_i) = \frac{e^{\beta_0 + \beta_1 \cdot x_i}}{1 + e^{\beta_0 + \beta_1 \cdot x_i}} \quad \text{and} \quad 1 - p(x_i) = \frac{1}{1 + e^{\beta_0 + \beta_1 \cdot x_i}}$$

then

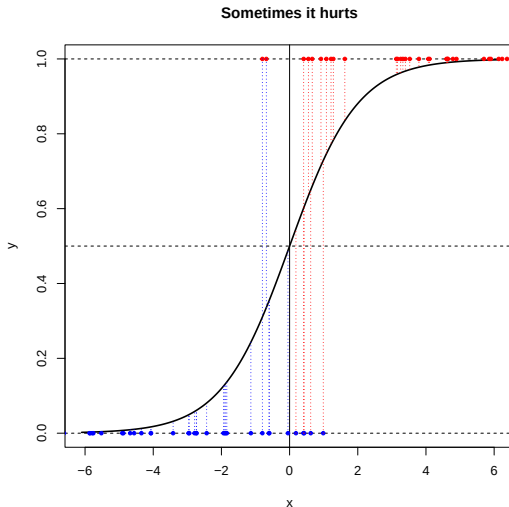
$$\frac{p(x_i)}{1 - p(x_i)} = e^{\beta_0 + \beta_1 \cdot x_i}$$

and thus

$$\log \left(\frac{p(x_i)}{1 - p(x_i)} \right) = \underbrace{\beta_0 + \beta_1 \cdot x_i}_{\text{linear part}} \overset{\text{Definition}}{=} \text{logit}(p(x_i))$$

So the logit model is equivalent to a log-linear model for the odds ratio $p(x_i)/1 - p(x_i)$.





Principle of maximum likelihood:

"Find the parameters of the theoretical distribution that maximize the probability of our given sample."

- Likelihood:

$$\begin{aligned}L(\beta_0, \beta_1) &= P_{\beta_0, \beta_1}(\{(x_1, y_1), \dots, (x_n, y_n)\}) \\&= P_{\beta_0, \beta_1}((x_1, y_1) \wedge \dots \wedge (x_n, y_n)) \\&= \prod_{i=1}^n P_{\beta_0, \beta_1}(x_i, y_i)\end{aligned}$$

- Max likelihood:

$$\arg_{\beta_0, \beta_1} \max L(\beta_0, \beta_1) = \arg_{\beta_0, \beta_1} \max \prod_{i=1}^n P_{\beta_0, \beta_1}(x_i, y_i)$$



So how compute $P_{\beta_0, \beta_1}((x_i, y_i))$ knowing that

$$P_{\beta_0, \beta_1}((x_i, y_i)) = \begin{cases} p(x_i) & \text{if } y_i = 1 \\ 1 - p(x_i) & \text{if } y_i = 0 \end{cases}$$



So how compute $P_{\beta_0, \beta_1}((x_i, y_i))$ knowing that

$$P_{\beta_0, \beta_1}((x_i, y_i)) = \begin{cases} p(x_i) & \text{if } y_i = 1 \\ 1 - p(x_i) & \text{if } y_i = 0 \end{cases}$$

Simple !

$$P_{\beta_0, \beta_1}((x_i, y_i)) = p(x_i)^{y_i} \cdot (1 - p(x_i))^{1-y_i}$$



So how compute $P_{\beta_0, \beta_1}((x_i, y_i))$ knowing that

$$P_{\beta_0, \beta_1}((x_i, y_i)) = \begin{cases} p(x_i) & \text{if } y_i = 1 \\ 1 - p(x_i) & \text{if } y_i = 0 \end{cases}$$

Simple !

$$P_{\beta_0, \beta_1}((x_i, y_i)) = p(x_i)^{y_i} \cdot (1 - p(x_i))^{1-y_i}$$

and then

$$L(\beta_0, \beta_1) = \prod_{i=1}^n p(x_i)^{y_i} \cdot (1 - p(x_i))^{1-y_i}$$

and then

$$\log L(\beta_0, \beta_1) = \sum_{i=1}^n y_i \cdot \log p(x_i) + (1 - y_i) \cdot \log(1 - p(x_i))$$



Generalization 1:

Many explanatory variables, two categories

Let a dataset with $p + 1$ variables y and $x_1, \dots, x_p$, where $x_i \in \mathbb{R}$ and $y \in \{0, 1\}$, and we search for f such $y = f(x_1, \dots, x_p)$.



Generalization 1:

Many explanatory variables, two categories

Let a dataset with $p + 1$ variables y and $x_1, \dots, x_p$, where $x_i \in \mathbb{R}$ and $y \in \{0, 1\}$, and we search for f such $y = f(x_1, \dots, x_p)$.

Remember when

$$P(y = 1|x) = p(x) = \frac{e^z}{1 + e^z}$$

and

$$P(y = 0|x) = 1 - p(x) = 1 - \frac{e^z}{1 + e^z}?$$



Generalization 1:

Many explanatory variables, two categories

Let a dataset with $p + 1$ variables y and $x_1, \dots, x_p$, where $x_i \in \mathbb{R}$ and $y \in \{0, 1\}$, and we search for f such $y = f(x_1, \dots, x_p)$.

Remember when

$$P(y = 1|x) = p(x) = \frac{e^z}{1 + e^z}$$

and

$$P(y = 0|x) = 1 - p(x) = 1 - \frac{e^z}{1 + e^z}?$$

But now

$$\begin{aligned} z &= \beta_0 + \beta_1 \cdot x_1 + \dots + \beta_p \cdot x_p \\ &= \beta_0 + \sum_{i=1}^p \beta_i \cdot x_i \end{aligned}$$

It is the only thing to change !



Generalization 2:

Many explanatory variables, many categories

Let a dataset with $p + 1$ variables y and $x_1, \dots, x_p$, where $x_i \in \mathbb{R}$ and $y \in \{1, \dots, K\}$, and we search for f such $y = f(x_1, \dots, x_p)$.

We form K linear regressions :

$$z_k = \beta_{0,k} + \beta_{1,k}x_1 + \beta_{2,k}x_2 + \dots + \beta_{p,k}x_p$$

and in a vector form :

$$z_k = \boldsymbol{\beta}_k^\tau \cdot \mathbf{X}$$

where

$$\boldsymbol{\beta}_k = \begin{pmatrix} \beta_{0,k} \\ \beta_{1,k} \\ \vdots \\ \beta_{p,k} \end{pmatrix} \text{ and } \mathbf{X} = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_p \end{pmatrix}$$

with $k \in \{1, \dots, K\}$.



Generalization 2:

Many explanatory variables, many categories

To process the multinomial logit model for K possible outcomes, we perform $K - 1$ independent binary regressions (binomial), in which one outcome is chosen as a "pivot" and then the other $K - 1$ outcomes are separately regressed against the pivot outcome.

This would proceed as follows, if outcome K (the last outcome) is chosen as the pivot:

$$\ln \frac{\Pr(y = 1)}{\Pr(y = K)} = \beta_1 \cdot \mathbf{X}$$

$$\ln \frac{\Pr(y = 2)}{\Pr(y = K)} = \beta_2 \cdot \mathbf{X}$$

.....

$$\ln \frac{\Pr(y = K - 1)}{\Pr(y = K)} = \beta_{K-1} \cdot \mathbf{X}$$

... Source.



Generalization 2:

Many explanatory variables, many categories

$$\Pr(y = 1) = \Pr(y = K) e^{\beta_1 \cdot \mathbf{X}}$$

$$\Pr(y = 2) = \Pr(y = K) e^{\beta_2 \cdot \mathbf{X}}$$

.....

$$\Pr(y = K - 1) = \Pr(y = K) e^{\beta_{K-1} \cdot \mathbf{X}}$$

And using the fact that all K of the probabilities must sum to one:

$$\Pr(y = K) = 1 - \sum_{k=1}^{K-1} \Pr(y = k) = 1 - \sum_{k=1}^{K-1} \Pr(y = K) e^{\beta_k \cdot \mathbf{X}}$$

we have

$$\Pr(y = K) = \frac{1}{1 + \sum_{k=1}^{K-1} e^{\beta_k \cdot \mathbf{X}}}$$



Generalization 2: Many explanatory variables, many categories

Which leads to

$$\Pr(y = 1) = \frac{e^{\beta_1 \cdot \mathbf{X}}}{1 + \sum_{k=1}^{K-1} e^{\beta_k \cdot \mathbf{X}}}$$

$$\Pr(y = 2) = \frac{e^{\beta_2 \cdot \mathbf{X}}}{1 + \sum_{k=1}^{K-1} e^{\beta_k \cdot \mathbf{X}}}$$

.....

$$\Pr(y = K - 1) = \frac{e^{\beta_{K-1} \cdot \mathbf{X}}}{1 + \sum_{k=1}^{K-1} e^{\beta_k \cdot \mathbf{X}}}$$

or more generally $i \in \{1, \dots, K\}$

$$\Pr(y = k) = \frac{e^{\beta_k \cdot \mathbf{X}}}{1 + \sum_{j=1}^{K-1} e^{\beta_j \cdot \mathbf{X}}}$$



Generalization 2: Many explanatory variables, many categories

Using

$$\Pr(y = k) = \frac{e^{\beta_k \cdot \mathbf{X}}}{1 + \sum_{j=1}^{K-1} e^{\beta_j \cdot \mathbf{X}}}$$

with $i \in \{1, \dots, K\}$, class is decided using :

$$\arg_k \max \Pr(y = k)$$



2 Main Algorithms

- Logistic Regression
- **Bayesian Supervised Classification**
- Support Vector Machine - SVM
- Decision Trees



The class of u is the class with the higher probability:

$$p_k(u) = P(y = k | \mathcal{X} = \mathbf{u}) \quad (1)$$



The class of u is the class with the higher probability:

$$p_k(u) = P(y = k | \mathcal{X} = \mathbf{u}) \quad (1)$$

$$= \frac{P(\mathcal{X} = \mathbf{u} | y = k) \cdot P(y = k)}{P(\mathcal{X} = \mathbf{u})} \quad (2)$$



The class of u is the class with the higher probability:

$$p_k(u) = P(y = k | \mathcal{X} = \mathbf{u}) \quad (1)$$

$$= \frac{P(\mathcal{X} = \mathbf{u} | y = k) \cdot P(y = k)}{P(\mathcal{X} = \mathbf{u})} \quad (2)$$

$$= \frac{P(\mathcal{X} = \mathbf{u} | y = k) \cdot P(y = k)}{\sum_{i=1}^K P(\mathcal{X} = \mathbf{u} | y = i) \cdot P(y = i)} \quad (3)$$



The class of u is the class with the higher probability:

$$p_k(u) = P(y = k | \mathcal{X} = \mathbf{u}) \quad (1)$$

$$= \frac{P(\mathcal{X} = \mathbf{u} | y = k) \cdot P(y = k)}{P(\mathcal{X} = \mathbf{u})} \quad (2)$$

$$= \frac{P(\mathcal{X} = \mathbf{u} | y = k) \cdot P(y = k)}{\sum_{i=1}^K P(\mathcal{X} = \mathbf{u} | y = i) \cdot P(y = i)} \quad (3)$$

$$\hat{y}(\mathbf{u}) = \arg \max_k p_k(\mathbf{u}) \quad (4)$$

Estimate $P(y = k)$ is easy, but estimate $P(\mathcal{X} = \mathbf{u} | y = k)$ is more difficult, and more precisely if P follows a parametric law, the estimations of the parameters is searched using the concept of maximum likelihood and can be hard.



For given sample $\{u_1, \dots, u_N\}$ and partition $P = (C_1, \dots, C_K)$, if we suppose that each set C_i is a sample of a random variable $\mathcal{X}_k$, with probability law $f_{\mathcal{X}_k}(\cdot|\theta_k)$, then for each set C_k the likelihood is given by:

$$L(C_k, \theta_k) = \prod_{u_i \in C_k} f_{\mathcal{X}_k}(u_i|\theta_k). \quad (5)$$

The classifying likelihood is given by

$$cl(P, \Theta) = \prod_{k=1}^K L(C_k, \theta_k). \quad (6)$$

Then the estimator of the vector of parameters $\hat{\Theta} = (\hat{\theta}_1, \dots, \hat{\theta}_k)$ is the one which maximizes the classifying likelihood:

$$\Theta = \arg_{\Theta} \max cl(P, \Theta)$$

This is the hard part.



With a given classification $P = C_1 \cup \dots \cup C_k$ of the training set.

M step (Maximization): Compute the estimators $(\pi_{\mathcal{X}_k}^n, \theta_k^n)$

$$\hat{\pi}_{\mathcal{X}_k} = P(y = k) = \frac{|C_k|}{N}, \quad k = 1, \dots, K \quad (7)$$

$$\Theta = \arg_{\Theta} \max cl(P, \Theta). \quad (8)$$

E step (Estimation): For $1 \leq i \leq n$ and $1 \leq k \leq K$

$$t_k^n(u_i) = \frac{P(\mathcal{X} = \mathbf{u} | y = k, \theta_k^n) \cdot P(y = k)}{\sum_{i=1}^K P(\mathcal{X} = \mathbf{u} | y = i, \theta_i^n) \cdot P(y = i)} \quad (9)$$

C step (Classification): Build clusters using maximal probability:

$$k^n(u_i) = \arg \max_{1 \leq k \leq K} t_k^n(u_i)$$



Bayesian probabilities:

$$P(\mathcal{X} = \mathbf{u} | y = k)$$



Bayesian probabilities:

$$\begin{aligned} P(\mathcal{X} = \mathbf{u} | y = k) \\ = P(\mathcal{X} = (u_1, \dots, u_p) | y = k) \end{aligned}$$



Bayesian probabilities:

$$\begin{aligned} P(\mathcal{X} = \mathbf{u} | y = k) \\ &= P(\mathcal{X} = (u_1, \dots, u_p) | y = k) \\ &= P(u_1, \dots, u_p | C_k) \end{aligned}$$



Bayesian probabilities:

$$\begin{aligned} P(\mathcal{X} = \mathbf{u} | y = k) \\ &= P(\mathcal{X} = (u_1, \dots, u_p) | y = k) \\ &= P(u_1, \dots, u_p | C_k) \\ &= P(u_1 | C_k) \cdot P(u_2 \dots, u_p | C_k, u_1) \end{aligned}$$



Bayesian probabilities:

$$\begin{aligned} P(\mathcal{X} = \mathbf{u} | y = k) \\ &= P(\mathcal{X} = (u_1, \dots, u_p) | y = k) \\ &= P(u_1, \dots, u_p | C_k) \\ &= P(u_1 | C_k) \cdot P(u_2 \dots, u_p | C_k, u_1) \\ &= P(u_1 | C_k) \cdot P(u_2 | C_k, u_1) \cdot P(u_3 \dots, u_p | C_k, u_1, u_2) \\ &= \dots \end{aligned}$$



Bayesian probabilities:

$$\begin{aligned} P(\mathcal{X} = \mathbf{u} | y = k) &= P(\mathcal{X} = (u_1, \dots, u_p) | y = k) \\ &= P(u_1, \dots, u_p | C_k) \\ &= P(u_1 | C_k) \cdot P(u_2 \dots, u_p | C_k, u_1) \\ &= P(u_1 | C_k) \cdot P(u_2 | C_k, u_1) \cdot P(u_3 \dots, u_p | C_k, u_1, u_2) \\ &= \dots \\ &= P(u_1 | C_k) \cdot P(u_2 | C_k, u_1) \dots P(u_p | C_k, u_1, \dots, u_{p-1}) \end{aligned}$$



Bayesian probabilities:

$$\begin{aligned}
 P(\mathcal{X} = \mathbf{u} | y = k) &= P(\mathcal{X} = (u_1, \dots, u_p) | y = k) \\
 &= P(u_1, \dots, u_p | C_k) \\
 &= P(u_1 | C_k) \cdot P(u_2 \dots, u_p | C_k, u_1) \\
 &= P(u_1 | C_k) \cdot P(u_2 | C_k, u_1) \cdot P(u_3 \dots, u_p | C_k, u_1, u_2) \\
 &= \dots \\
 &= P(u_1 | C_k) \cdot P(u_2 | C_k, u_1) \dots P(u_p | C_k, u_1, \dots, u_{p-1})
 \end{aligned}$$

Naive Bayes Hypothesis:

$$\begin{aligned}
 P(\mathcal{X} = \mathbf{u} | C_k) &= P(u_1 | C_k) \cdot P(u_2 | C_k, u_1) \dots P(u_p | C_k, u_1, \dots, u_{p-1}) \\
 &= P(u_1 | C_k) \cdot P(u_2 | C_k, \cancel{u_1}) \dots P(u_p | C_k, \cancel{u_1}, \dots, \cancel{u_{p-1}})
 \end{aligned}$$



Bayesian probabilities:

$$\begin{aligned}
 P(\mathcal{X} = \mathbf{u} | y = k) &= P(\mathcal{X} = (u_1, \dots, u_p) | y = k) \\
 &= P(u_1, \dots, u_p | C_k) \\
 &= P(u_1 | C_k) \cdot P(u_2 \dots, u_p | C_k, u_1) \\
 &= P(u_1 | C_k) \cdot P(u_2 | C_k, u_1) \cdot P(u_3 \dots, u_p | C_k, u_1, u_2) \\
 &= \dots \\
 &= P(u_1 | C_k) \cdot P(u_2 | C_k, u_1) \dots P(u_p | C_k, u_1, \dots, u_{p-1})
 \end{aligned}$$

Naive Bayes Hypothesis:

$$\begin{aligned}
 P(\mathcal{X} = \mathbf{u} | C_k) &= P(u_1 | C_k) \cdot P(u_2 | C_k, u_1) \dots P(u_p | C_k, u_1, \dots, u_{p-1}) \\
 &= P(u_1 | C_k) \cdot P(u_2 | C_k, \cancel{u_1}) \dots P(u_p | C_k, \cancel{u_1, \dots, u_{p-1}}) \\
 &= P(u_1 | C_k) \cdot P(u_2 | C_k) \dots P(u_p | C_k)
 \end{aligned}$$



In the computation of

$$P(\mathcal{X} = \mathbf{u} | C_k) = P_{\theta_1}(u_1 | C_k) \cdot P_{\theta_2}(u_2 | C_k) \cdots P_{\theta_p}(u_p | C_k)$$

if all variables are Gaussian

$$P_{\theta_i}(u_i | C_k) = \frac{1}{\sqrt{2\pi\sigma_{i,k}^2}} e^{-\frac{(u_i - \mu_{i,k})^2}{2\sigma_{i,k}^2}}$$

then the parameter estimation step, to find the solutions $\hat{\theta}_k = (\hat{\mu}_k, \hat{\sigma}_k)$, with $\hat{\mu}_k = (\mu_{1,k}, \dots, \mu_{p,k})$ and $\hat{\theta}_k = (\theta_{1,k}, \dots, \theta_{p,k})$, of the maximum likelihood problem, for each class is very easy:

$$\mu_{i,k} = \frac{1}{|C_k|} \sum_{u_i | \mathbf{u} \in C_k} u_i$$

and

$$\sigma_{i,k}^2 = \frac{1}{|C_k|} \sum_{u_i | \mathbf{u} \in C_k} (u_i - \mu_{i,k})^2$$



A “less naive” hypothesis is to suppose that data follow multivariate Gaussian distributions, then:

$$P(\mathbf{u}|C_k) = \frac{1}{\sqrt{(2\pi)^p |\Sigma_k|}} \exp \left(-\frac{1}{2} (\mathbf{u} - \mu_k)^\tau \Sigma_k^{-1} (\mathbf{u} - \mu_k) \right),$$

where:

- $\mathbf{u} = (u_1, \dots, u_p)$,
- μ_k is the mean vector for the class C_k ,
- Σ_k is the covariance matrix for the class C_k , and $|\Sigma_k|$ its determinant.

Again, in this case, the estimation is easy and the parameters are given by $\hat{\mu}_k = (\mu_{1,k}, \dots, \mu_{p,k})$ and $\hat{\theta}_k = \Sigma_k$.

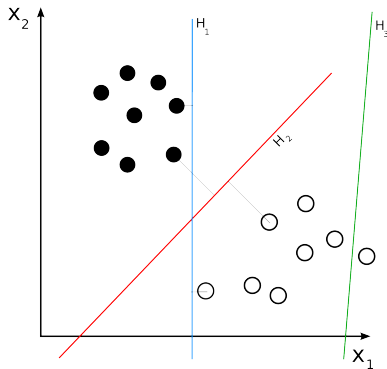


2 Main Algorithms

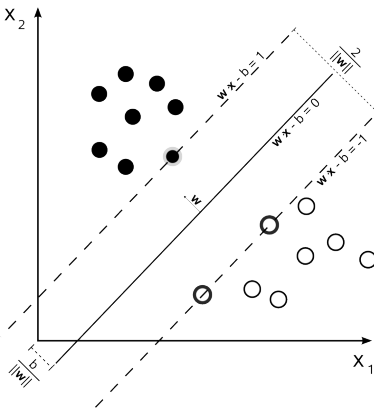
- Logistic Regression
- Bayesian Supervised Classification
- **Support Vector Machine - SVM**
- Decision Trees



First idea of SVM : Find a linear hyperplane to separate two classes...



First idea of SVM : Find a linear hyperplane to separate two classes...



..but with a maximum margin.

We are given some training data $\mathcal{D}$, a set of n points of the form :

$$\mathcal{D} = \{(\mathbf{x}_i, y_i) \mid \mathbf{x}_i \in \mathbb{R}^p, y_i \in \{-1, 1\}\}_{i=1}^n$$

We search for a linear model

$$\hat{y} = h(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} - b$$

such

- margins must follow : $\mathbf{w} \cdot \mathbf{x} - b = 1$, and $\mathbf{w} \cdot \mathbf{x} - b = -1$,
- general separation hyperplane equation $\mathbf{w} \cdot \mathbf{x} - b = 0$,
- geometry tell us that the distance between a point and this last plane is given by

$$\frac{||\mathbf{w} \cdot \mathbf{x} - b||}{||\mathbf{w}||}$$

- then the distance between the two margins is given by $\frac{2}{||\mathbf{w}||}$, so we want to minimize $||\mathbf{w}||$,

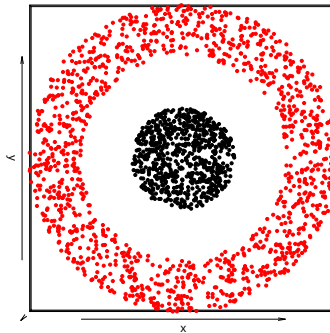


- we define a function $h(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} - b$ such $h(\mathbf{x}_i) \geq 1$ if $\mathbf{x}_i$ is of the first class, and $h(\mathbf{x}_i) \leq -1$ if $\mathbf{x}_i$ is of the second,
- in other words $y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \geq 1$,
- thus we want to minimize $\frac{1}{2}\|\mathbf{w}\|^2$ with variables $\mathbf{w}, b$, subject to constraints $y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \geq 1$
- which can be solved using Lagrange multipliers α_i :

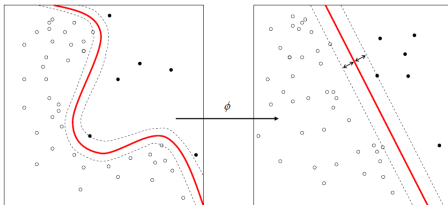
$$\min_{\mathbf{w}, b, \alpha} \left\{ \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i [y_i(\mathbf{w} \cdot \mathbf{x}_i - b) - 1] \right\}.$$



The Kernel Trick



Find a function $\Phi : V \rightarrow W$ which map the data from the original space V where they are not linearly separable into another space W where they will be:



But we don't need to know $\Phi(x)$ and $\Phi(y)$ for all x and y , only their inner product:

$$K(x, y) = \langle \Phi(x), \Phi(y) \rangle_V$$

The most classical kernel is the gaussian kernel :

$$K(\mathbf{x}, \mathbf{y}) = \exp \left(-\frac{\|\mathbf{x} - \mathbf{y}\|^2}{2\sigma^2} \right).$$



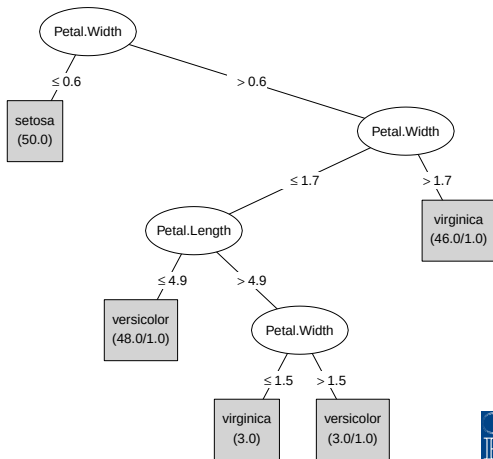
2 Main Algorithms

- Logistic Regression
- Bayesian Supervised Classification
- Support Vector Machine - SVM
- Decision Trees



At each stage chose the variable (and a value of this latter) which divide the considered group into more homogeneous subgroups.

C4.5 Decision Tree for Iris Data





- Let a message containing N symbols to distinguish.



- Let a message containing N symbols to distinguish.
- If $N = 2^k$, then $k = \log_2(N)$ is quantity of information needed to distinguish the N symbols.



- Let a message containing N symbols to distinguish.
- If $N = 2^k$, then $k = \log_2(N)$ is quantity of information needed to distinguish the N symbols.
- Suppose that symbols are grouped in n categories (ex. characters) such $N = N_1 + \dots + N_n$, where N_i is the size of the i th category.



- Let a message containing N symbols to distinguish.
- If $N = 2^k$, then $k = \log_2(N)$ is quantity of information needed to distinguish the N symbols.
- Suppose that symbols are grouped in n categories (ex. characters) such $N = N_1 + \dots + N_n$, where N_i is the size of the i th category.
- The quantity of information to distinguish a symbol of the i th category is given by $\log_2(N_i)$.



- Let a message containing N symbols to distinguish.
- If $N = 2^k$, then $k = \log_2(N)$ is quantity of information needed to distinguish the N symbols.
- Suppose that symbols are grouped in n categories (ex. characters) such $N = N_1 + \dots + N_n$, where N_i is the size of the i th category.
- The quantity of information to distinguish a symbol of the i th category is given by $\log_2(N_i)$.
- The probability to belong to the i th category is simply given by $P_i = N_i/N$.



- Let a message containing N symbols to distinguish.
- If $N = 2^k$, then $k = \log_2(N)$ is quantity of information needed to distinguish the N symbols.
- Suppose that symbols are grouped in n categories (ex. characters) such $N = N_1 + \dots + N_n$, where N_i is the size of the i th category.
- The quantity of information to distinguish a symbol of the i th category is given by $\log_2(N_i)$.
- The probability to belong to the i th category is simply given by $P_i = N_i/N$.
- Then the mean quantity of information to distinguish a symbol knowing its category is given by

$$\sum_{i=1}^n P_i \log_2(N_i).$$



- Distinguish a symbol can be made in two steps:



- Distinguish a symbol can be made in two steps:
 - 1 distinguish its category X ,



- Distinguish a symbol can be made in two steps:
 - 1 distinguish its category X ,
 - 2 distinguish it knowing its category.



- Distinguish a symbol can be made in two steps:
 - 1 distinguish its category X ,
 - 2 distinguish it knowing its category.
- The amount of information to distinguish a symbol from the message is given by $\log_2(N)$, but is also the sum of the quantity of information $H(X)$ to determine its category and the mean quantity of information to distinguish a symbol knowing its category:

$$\log_2(N) = H(X) + \sum_i P_i \log_2(N_i)$$



- Distinguish a symbol can be made in two steps:
 - 1 distinguish its category X ,
 - 2 distinguish it knowing its category.
- The amount of information to distinguish a symbol from the message is given by $\log_2(N)$, but is also the sum of the quantity of information $H(X)$ to determine its category and the mean quantity of information to distinguish a symbol knowing its category:

$$\log_2(N) = H(X) + \sum_i P_i \log_2(N_i)$$

then

$$H(X) = \log_2(N) - \sum_i P_i \log_2(N_i) \quad (11)$$

$$= - \sum_i P_i \log_2(N_i/N) \quad (12)$$

$$= - \sum_i P_i \log_2(P_i)$$



Let S a set of elements with several classes possible for elements of S . The entropy of S is given by:

$$H(S) = - \sum_{i=1}^N P(i) \log_2 P(i)$$

Where:

- N is the number of classes of S ,
- $P(i)$ is the probability or the frequency of the i th class in the subset S .

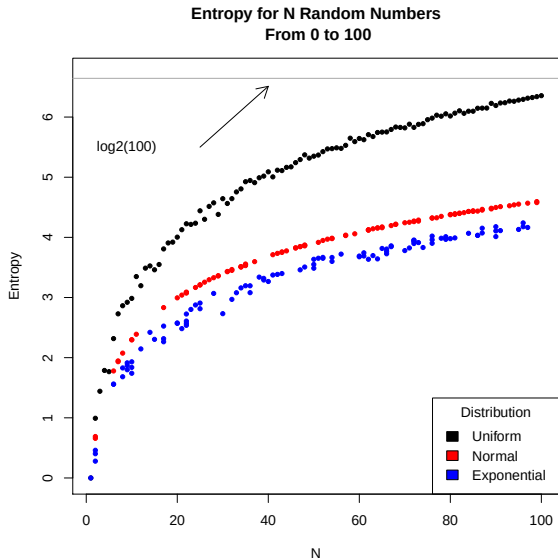


- 1 $H(S) \geq 0$.
- 2 $H(S) = 0$ only when S is a perfectly classified subset with only one class.
- 3 $H(S) \leq \log_2(N)$, with equality if and only if P is uniform, i.e., $P(i) = 1/N$.

An illustration of the last point is given when $P(i) = 1/N$:

$$\begin{aligned} H(X) &= - \sum_i P(i) \log_2(P(i)) \\ &= - \sum_i \frac{1}{N} \log_2\left(\frac{1}{N}\right) \\ &= - \log_2\left(\frac{1}{N}\right) \\ &= \log_2(N) \end{aligned}$$





ID3 (Iterative Dichotomiser 3) is an algorithm invented by Ross Quinlan based on information entropy. Let the Golf Weather example (the class to predict is “Play”):

	Outlook	Temperature	Humidity	Wind	Play
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No



The starting point : computing the entropy (uncertainty) of the set

- 14 items in the whole set
- 9 playing cases
- 5 non playing cases

Entropy of the whole set based on the playing cases (the target class):

$$E(S) = -\frac{9}{14} * \log_2\left(\frac{9}{14}\right) - \frac{5}{14} * \log_2\left(\frac{5}{14}\right) = 0.94$$

Let define $Info(S) = E(S)$ as a measure of the quantity of information needed (or uncertainty) to determine the class of an element of S .





- Suppose now that we know the value of an attribute T (let say Outlook for the sake of illustration).



- Suppose now that we know the value of an attribute T (let say Outlook for the sake of illustration).
- For each value T_i of T we can compute $Info(T_i)$ based on the playing cases.



- Suppose now that we know the value of an attribute T (let say Outlook for the sake of illustration).
- For each value T_i of T we can compute $Info(T_i)$ based on the playing cases.
- The mean value of information (uncertainty) knowing the values of T for a given element is equal to:

$$Info(T, S) = \sum_i \frac{|T_i|}{|T|} Info(T_i)$$



- Suppose now that we know the value of an attribute T (let say Outlook for the sake of illustration).
- For each value T_i of T we can compute $Info(T_i)$ based on the playing cases.
- The mean value of information (uncertainty) knowing the values of T for a given element is equal to:

$$Info(T, S) = \sum_i \frac{|T_i|}{|T|} Info(T_i)$$

- We can then define the decrease of uncertainty as gain of certainty using attribute T :

$$Gain(T, S) = Info(S) - Info(T, S)$$



- Suppose now that we know the value of an attribute T (let say Outlook for the sake of illustration).
- For each value T_i of T we can compute $Info(T_i)$ based on the playing cases.
- The mean value of information (uncertainty) knowing the values of T for a given element is equal to:

$$Info(T, S) = \sum_i \frac{|T_i|}{|T|} Info(T_i)$$

- We can then define the decrease of uncertainty as gain of certainty using attribute T :

$$Gain(T, S) = Info(S) - Info(T, S)$$

- Attribute T with the greatest gain is a good choice to split the set and decrease uncertainty!



Lets compute uncertainty for each attribut starting with "Outlook"



Lets compute uncertainty for each attribut starting with "Outlook"

- $E(Outlook = Sunny) = -2/5 * \log_2(2/5) - 3/5 * \log_2(3/5) = 0.9710$



Lets compute uncertainty for each attribut starting with "Outlook"

- $E(Outlook = Sunny) = -2/5 * \log_2(2/5) - 3/5 * \log_2(3/5) = 0.9710$
- $E(Outlook = Overcast) = -4/4 * \log_2(4/4) - 0 * \log_2(0) = 0$



Lets compute uncertainty for each attribut starting with "Outlook"

- $E(Outlook = Sunny) = -2/5 * \log_2(2/5) - 3/5 * \log_2(3/5) = 0.9710$
- $E(Outlook = Overcast) = -4/4 * \log_2(4/4) - 0 * \log_2(0) = 0$
- $E(Outlook = Rain) = -3/5 * \log_2(3/5) - 2/5 * \log_2(2/5) = 0.9710$



Lets compute uncertainty for each attribut starting with "Outlook"

- $E(Outlook = Sunny) = -2/5 * \log_2(2/5) - 3/5 * \log_2(3/5) = 0.9710$
- $E(Outlook = Overcast) = -4/4 * \log_2(4/4) - 0 * \log_2(0) = 0$
- $E(Outlook = Rain) = -3/5 * \log_2(3/5) - 2/5 * \log_2(2/5) = 0.9710$
- $Info(Outlook, S) = 5/14 * 0.9710 + 4/14 * 0 + 5/14 * 0.9710 = 0.694$



Lets compute uncertainty for each attribut starting with "Outlook"

- $E(\text{Outlook} = \text{Sunny}) = -2/5 * \log_2(2/5) - 3/5 * \log_2(3/5) = 0.9710$
- $E(\text{Outlook} = \text{Overcast}) = -4/4 * \log_2(4/4) - 0 * \log_2(0) = 0$
- $E(\text{Outlook} = \text{Rain}) = -3/5 * \log_2(3/5) - 2/5 * \log_2(2/5) = 0.9710$
- $\text{Info}(\text{Outlook}, S) = 5/14 * 0.9710 + 4/14 * 0 + 5/14 * 0.9710 = 0.694$
- $\text{Gain}(\text{Outlook}, S) = 0.94 - 0.694 = 0.246$



Lets compute uncertainty for each attribut starting with “Outlook”

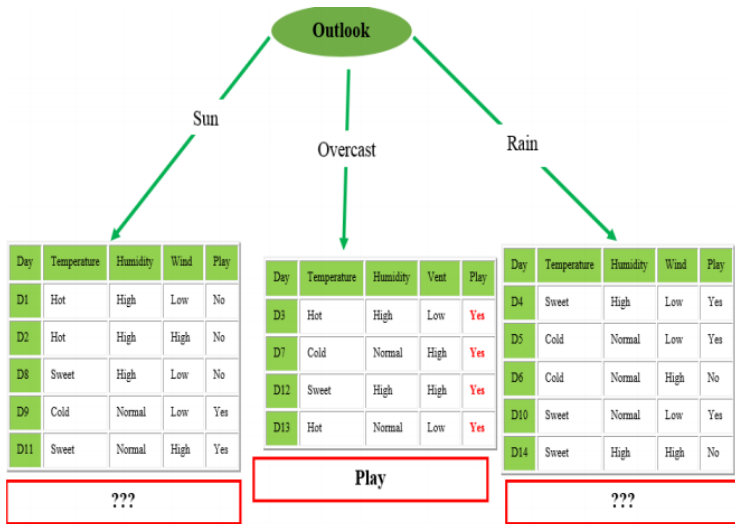
- $E(Outlook = Sunny) = -2/5 * \log_2(2/5) - 3/5 * \log_2(3/5) = 0.9710$
- $E(Outlook = Overcast) = -4/4 * \log_2(4/4) - 0 * \log_2(0) = 0$
- $E(Outlook = Rain) = -3/5 * \log_2(3/5) - 2/5 * \log_2(2/5) = 0.9710$
- $Info(Outlook, S) = 5/14 * 0.9710 + 4/14 * 0 + 5/14 * 0.9710 = 0.694$
- $Gain(Outlook, S) = 0.94 - 0.694 = 0.246$
- $Gain(Wind, S) = 0.048$
- $Gain(Temperature, S) = 0.0289$
- $Gain(Humidity, S) = 0.1515$

Attribute “Outlook” is best choice to decrease uncertainty...



Splitting the set using the “Outlook” attribute:

	Outlook	Temperature	Humidity	Wind	Play
3	Overcast	Hot	High	Weak	Yes
7	Overcast	Cool	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
14	Rain	Mild	High	Strong	No
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes



> weather.tree

node), split, n, deviance, yval, (yprob)

* denotes terminal node

1) root 14 18.250 Yes (0.3571 0.6429)

2) Outlook: Rain,Sunny 10 13.860 No (0.5000 0.5000)

4) Humidity: High 5 5.004 No (0.8000 0.2000) *

5) Humidity: Normal 5 5.004 Yes (0.2000 0.8000) *

3) Outlook: Overcast 4 0.000 Yes (0.0000 1.0000) *



Then the general algorithm of ID3 is the following:

- Take all unused variables and count their gain,
- Split on variable for which information gain is maximum,
- Repeat for each subset of the splitting.



- The choice of split is based on gain, based on uncertainty for an attribute

$$Info(T, S) = \sum_i \frac{|T_i|}{|T|} Info(T_i) \quad (14)$$



- The choice of split is based on gain, based on uncertainty for an attribute

$$Info(T, S) = \sum_i \frac{|T_i|}{|T|} Info(T_i) \quad (14)$$

- Uncertainty for a set for a given value of an attribute is given by

$$Info(T_i) = E(T_i) = - \sum_{j=1}^n f_{T_i}(j) \log_2 f_{T_i}(j) \quad (15)$$



- The choice of split is based on gain, based on uncertainty for an attribute

$$Info(T, S) = \sum_i \frac{|T_i|}{|T|} Info(T_i) \quad (14)$$

- Uncertainty for a set for a given value of an attribute is given by

$$Info(T_i) = E(T_i) = - \sum_{j=1}^n f_{T_i}(j) \log_2 f_{T_i}(j) \quad (15)$$

- If an attribute have a very large number of values or, worst, if an attribute has a different value for each “individual” then

$$Info(T, S) = \sum_i \frac{1}{|T|} Info(T_i)$$



- The choice of split is based on gain, based on uncertainty for an attribute

$$Info(T, S) = \sum_i \frac{|T_i|}{|T|} Info(T_i) \quad (14)$$

- Uncertainty for a set for a given value of an attribute is given by

$$Info(T_i) = E(T_i) = - \sum_{j=1}^n f_{T_i}(j) \log_2 f_{T_i}(j) \quad (15)$$

- If an attribute have a very large number of values or, worst, if an attribute has a different value for each “individual” then

$$Info(T, S) = \sum_i \frac{1}{|T|} Info(T_i) = \sum_i \frac{1}{|T|} .0 = 0 \quad (16)$$



- The choice of split is based on gain, based on uncertainty for an attribute

$$Info(T, S) = \sum_i \frac{|T_i|}{|T|} Info(T_i) \quad (14)$$

- Uncertainty for a set for a given value of an attribute is given by

$$Info(T_i) = E(T_i) = - \sum_{j=1}^n f_{T_i}(j) \log_2 f_{T_i}(j) \quad (15)$$

- If an attribute have a very large number of values or, worst, if an attribute has a different value for each “individual” then

$$Info(T, S) = \sum_i \frac{1}{|T|} Info(T_i) = \sum_i \frac{1}{|T|} .0 = 0 \quad (16)$$

- This attribute will be chosen as the first choice, but it's a bad choice.



The C4.5 algorithm is a refinement of ID3 proposed by Quinlan:

- The choice of split is based on gain ratio:

$$GainRatio(T, S) = \frac{Gain(T, S)}{SplitInfo(T, S)}$$



The C4.5 algorithm is a refinement of ID3 proposed by Quinlan:

- The choice of split is based on gain ratio:

$$GainRatio(T, S) = \frac{Gain(T, S)}{SplitInfo(T, S)}$$

- where $SplitInfo(T, S)$ is

$$SplitInfo(T, S) = - \sum_{i=1}^n P'(j|T) \log_2 P'(j|T)$$

where $P'(j|T)$ the proportion of elements of attribute T with the j -th value.



- For the sake of illustration if attribute T has a unique value for the n “individuals” then

$$SplitInfo(T, S) = - \sum_{i=1}^n \frac{1}{n} \log_2 \frac{1}{n} = \log_2 n$$



- For the sake of illustration if attribute T has a unique value for the n “individuals” then

$$SplitInfo(T, S) = - \sum_{i=1}^n \frac{1}{n} \log_2 \frac{1}{n} = \log_2 n$$

while if T has k different values with subsets of size q (i.e. $n = k.q$), then

$$SplitInfo(T, S) = - \sum_{i=1}^k \frac{q}{n} \log_2 \frac{q}{n} = \log_2 k < \log_2 n$$



- 1 Introduction
- 2 Main Algorithms
- 3 Validation



3 Validation

- Prediction Quality
- Minimizing the Random Effect



Question : How to measure the quality of a predictive model?



Question : How to measure the quality of a predictive model?

The starting point: the confusion matrix

	Predicted Positive	Predicted Negative
Really Positive	TP	FN
Really Negative	FP	TN

Remark:

The confusion matrix is defined for a two classes problem but can be extended for more classes.



	Predicted Positive	Predicted Negative
Really Positive	TP	FN
Really Negative	FP	TN

Measures:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad \% \text{ of total good predictions}$$

$$\text{Error Rate} = \frac{FP + FN}{TP + TN + FP + FN} \quad \% \text{ of bad predictions}$$



	Predicted Positive	Predicted Negative
Really Positive	TP	FN
Really Negative	FP	TN

$$\text{Recall} = \frac{TP}{TP + FN} \quad \% \text{ of good positive vs expected pred.}$$

$$\text{Specificity} = \frac{TN}{TN + FP} \quad \% \text{ of good negative vs expected pred.}$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad \% \text{ of good vs given pred.}$$

Recall is also called Sensitivity.



	Predicted Positive	Predicted Negative
Really Positive	TP	FN
Really Negative	FP	TN

A point of the ROC space is given by :

$$\begin{aligned} (\text{Recall}, 1-\text{specificity}) &= \left(\frac{TP}{TP + FN}, \frac{FP}{FP + TN} \right) \\ &= (\text{TPR}, \text{FPR}) \end{aligned}$$

with TPR = True Positive Rate , and FPR = False Positive Rate.

ROC is the abbreviation for Receiver Operating Characteristic, and was first used during World War II for the analysis of radar signals before it was employed in signal detection theory and classification (source Wikipedia).



	Predicted Positive	Predicted Negative
Really Positive	TP	FN
Really Negative	FP	TN

$\left(\frac{TP}{TP + FN}, \frac{FP}{FP + TN} \right)$	ROC signification
(0, 0)	classifier always say "negative"
(1, 1)	classifier always say "positive"
(1, 0)	classifier is always right
(0.5, 0.5)	classifier is random
(0, 1)	classifier is always wrong ¹

¹If we inverse it then everything is OK.

Example : Case A

TP = 63	FN = 37	100
FP = 28	TN = 72	100
91	109	200

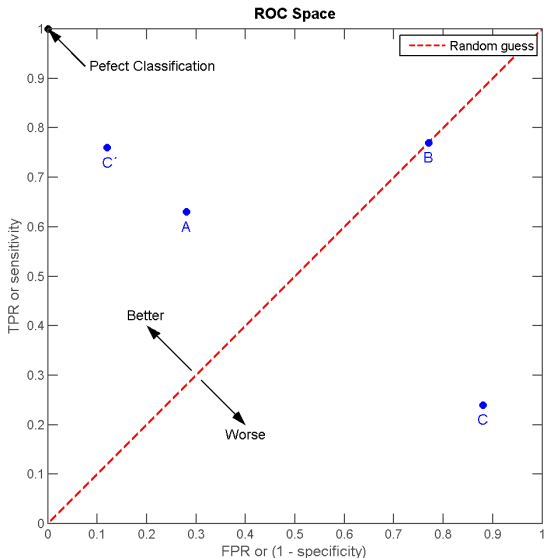
$$\text{Accuracy} = \frac{63 + 72}{200} = 0.675$$

$$\text{Precision} = \frac{63}{91} = 0.692$$

$$\left(\frac{TP}{TP + FN}, \frac{FP}{FP + TN} \right) = (0.63, 0.28)$$

Source Wikipedia.





Example : Case B

TP = 77	FN = 23	100
FP = 77	TN = 23	100
154	46	200

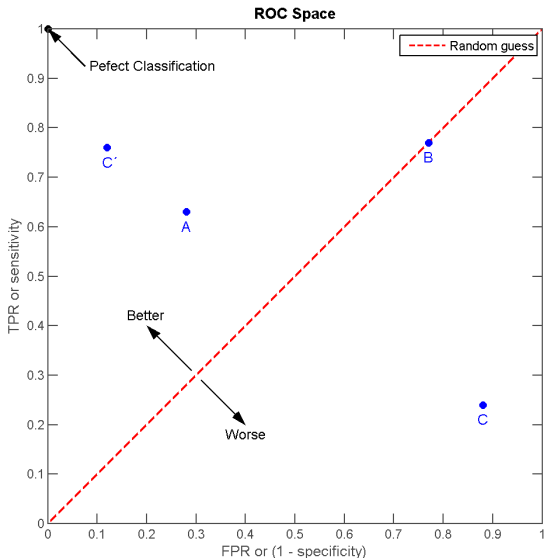
$$\text{Accuracy} = \frac{77 + 23}{200} = 0.50$$

$$\text{Precision} = \frac{77}{154} = 0.50$$

$$\left(\frac{TP}{TP + FN}, \frac{FP}{FP + TN} \right) = (0.77, 0.77)$$

Source Wikipedia.





Example : Case C

TP = 24	FN = 76	100
FP = 88	TN = 12	100
112	88	200

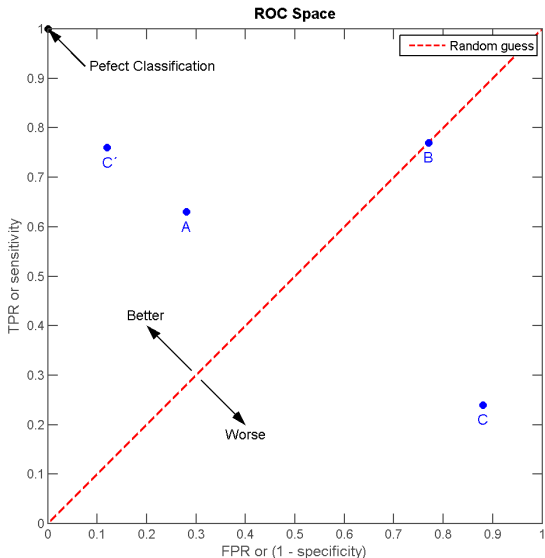
$$\text{Accuracy} = \frac{24 + 12}{200} = 0.18$$

$$\text{Precision} = \frac{24}{112} = 0.21$$

$$\left(\frac{TP}{TP + FN}, \frac{FP}{FP + TN} \right) = (0.24, 0.88)$$

Source Wikipedia.





Example : Case C'

TP = 76	FN = 24	100
FP = 12	TN = 88	100
88	112	200

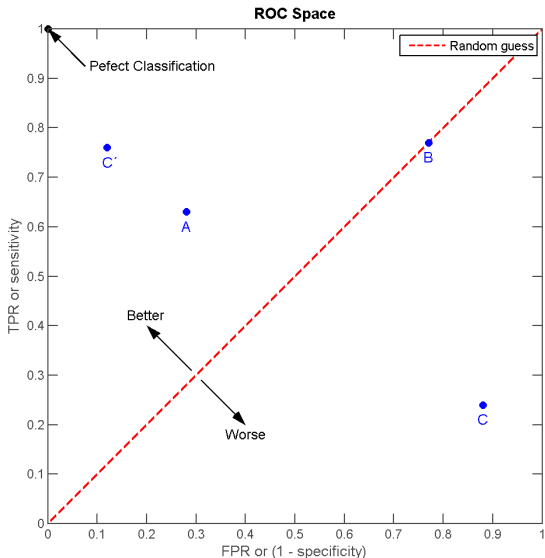
$$\text{Accuracy} = \frac{76 + 88}{200} = 0.82$$

$$\text{Precision} = \frac{76}{88} = 0.86$$

$$\left(\frac{TP}{TP + FN}, \frac{FP}{FP + TN} \right) = (0.76, 0.12)$$

Source Wikipedia.

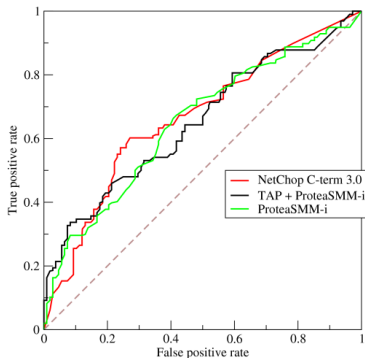




If we can decide the class based on a score s such for a given threshold parameter S :

- $s < S \Rightarrow$ positive,
- $s \geq S \Rightarrow$ negative,

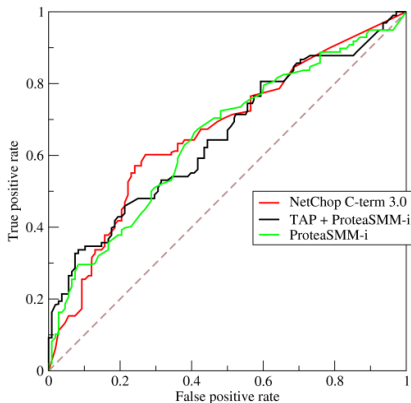
Then for using all possible values for S we can plot the ROC curve:



Source Wikipedia.



The Area Under the Curve (called AUC) provides a measure of quality for a classifier: an area of 1 represents a perfect test; an area of .5 represents a worthless test.



Source Wikipedia.



ROC does not take into account the random effect, to correct this, the Cohen's kappa compares the Observed Accuracy with the Expected Accuracy (random chance):

$$\kappa = \frac{P(a) - P(e)}{1 - P(e)}$$

where :

- $P(a)$ is the observed accuracy,
- $P(e)$ is the expected accuracy.



Example : Case A

TP = 63	FN = 37	100
FP = 28	TN = 72	100
91	109	200

Probabilities

$\frac{63}{200} = 0.315$	$\frac{37}{200} = 0.185$	$\frac{100}{200} = 0.5$
$\frac{28}{200} = 0.14$	$\frac{72}{200} = 0.36$	$\frac{100}{200} = 0.5$
$\frac{91}{200} = 0.455$	$\frac{109}{200} = 0.545$	1



Example : Case A Probabilities

0.315	0.185	0.5
0.14	0.36	0.5
0.455	0.545	1

$$\text{Observed Accuracy} = 0.315 + 0.36 = 0.675$$

$$\text{Expected Accuracy} = 0.5 * 0.455 + 0.5 * 0.545 = 0.500$$

$$\kappa = \frac{0.675 - 0.500}{1 - 0.500} = \frac{0.275}{0.500} = 0.55$$



Example : Case B Probabilities

0.385	0.115	0.5
0.385	0.115	0.5
0.77	0.23	1

$$\text{Observed Accuracy} = 0.385 + 0.115 = 0.500$$

$$\text{Expected Accuracy} = 0.5 * 0.77 + 0.5 * 0.23 = 0.500$$

$$\kappa = \frac{0.500 - 0.500}{1 - 0.500} = \frac{0}{0.500} = 0$$



Example : Case C Probabilities

0.12	0.38	0.5
0.44	0.06	0.5
0.56	0.44	1

$$\text{Observed Accuracy} = 0.12 + 0.06 = 0.18$$

$$\text{Expected Accuracy} = 0.5 * 0.56 + 0.5 * 0.44 = 0.500$$

$$\kappa = \frac{0.18 - 0.500}{1 - 0.500} = \frac{-0.32}{0.500} = -0.64$$



Example : Case C' Probabilities

0.38	0.12	0.5
0.06	0.44	0.5
0.44	0.56	1

$$\text{Observed Accuracy} = 0.38 + 0.44 = 0.82$$

$$\text{Expected Accuracy} = 0.5 * 0.44 + 0.5 * 0.56 = 0.500$$

$$\kappa = \frac{0.82 - 0.500}{1 - 0.500} = \frac{0.32}{0.500} = 0.64$$



	$\hat{C}_1$	$\hat{C}_2$	$\dots$	$\hat{C}_{n-1}$	$\hat{C}_n$
C_1	✓	✗	$\dots$	✗	✗
C_2	✗	✓	$\vdots$	✗	✗
$\vdots$	✗	$\dots$	$\vdots$	$\dots$	✗
C_{n-1}	✗	✗	$\vdots$	✓	✗
C_n	✗	✗	$\dots$	✗	✓

Where C_i is the real class i and $\hat{C}_i$ is the predicted one.

$$\text{Accuracy} = \frac{\sum \checkmark}{\sum \checkmark + \sum \times} \quad \% \text{ of total good predictions}$$

$$\text{Error Rate} = \frac{\sum \times}{\sum \checkmark + \sum \times} \quad \% \text{ of bad predictions}$$

1 Introduction

2 Main Algorithms

3 Validation

- Prediction Quality
- Minimizing the Random Effect



How to be sure that the good prediction is not due to a random effect? (a good test test)



How to be sure that the good prediction is not due to a random effect? (a good test test)

Training Set and Test Set



How to be sure that the good prediction is not due to a random effect? (a good test test)

Training Set and Test Set

- Divide the set of known groups in two sets:



How to be sure that the good prediction is not due to a random effect? (a good test test)

Training Set and Test Set

- Divide the set of known groups in two sets:
 - 1 the Training set,



How to be sure that the good prediction is not due to a random effect? (a good test test)

Training Set and Test Set

- Divide the set of known groups in two sets:
 - 1 the Training set,
 - 2 the Test set.



How to be sure that the good prediction is not due to a random effect? (a good test test)

Training Set and Test Set

- Divide the set of known groups in two sets:
 - 1 the Training set,
 - 2 the Test set.
- Use the Training set to “tune the model”.



How to be sure that the good prediction is not due to a random effect? (a good test test)

Training Set and Test Set

- Divide the set of known groups in two sets:
 - 1 the Training set,
 - 2 the Test set.
- Use the Training set to “tune the model”.
- Predict the groups for the Test set and compare with the true values.





Training Set

Test Set



K-fold cross-validation



Training Set

Test Set



Leave-one-out



Training Set

Test Set



How to validate a model on a whole sample of data? In all cases : split the given data set in 2 subsets: the training set and the validation set:

- Repeated random sub-sampling validation: randomly splits the dataset into training and validation sets.
- K-fold cross-validation: the original sample is randomly partitioned into K subsamples. Of the K subsamples, a single subsample is retained as the validation data for testing the model, and the remaining K -1 subsamples are used as training data. The cross-validation process is then repeated K times (the folds), with each of the K subsamples used exactly once as the validation data.
- Leave-one-out cross-validation: using a single observation from the original sample as the validation data, and the remaining observations as the training data. This is repeated such that each observation in the sample is used once as the validation data.

